

json_serializable

- Lots of errors before the code is generated
- Incomprehensible code is generated

What are the benefits of using code generation?

You may be wondering: "If code generation is optional in Riverpod, why use it?"

As always with packages: To make your life easier. This includes but is not limited to:

- better syntax, more readable/flexible, and with a reduced learning curve.
 - No need to worry about the type of provider. Write your logic, and Riverpod will pick the most suitable provider for you.
 - The syntax no longer looks like we're defining a "dirty global variable". Instead we are defining a custom function/class.
 - Passing parameters to providers is now unrestricted. Instead of being limited to using [.family](#) and passing a single positional parameter, you can now pass any parameter. This includes named parameters, optional ones, and even default values.
- **stateful hot-reload** of the code written in Riverpod.
- better debugging, through the generation of extra metadata that the debugger then picks up.
- some Riverpod features will be available only with code generation.

```
final helloProvider = Provider<String>((ref) {  
    return 'Hello';  
});
```

```
@Riverpod(keepAlive: true)  
String hello>HelloRef ref) {  
    return 'Hello';  
}
```